DESIGN DOCUMENT · ANGULAR COMPONENT

Damoh Reports — Architecture & Design

High-level and low-level design for `DamohReportsComponent`, the multi-form inspection report viewer and Excel exporter used across ParakhWeb's Super Admin reporting surface.

Module super-admin / reports / damoh-reports

Type Angular 15+ component (standalone-style module component)

Forms covered 10 report types (11 form IDs) **As of** 2026-09-01

Source basis: this workspace is not a Git repository, so no commit history was available to derive intent or change timeline from. Everything below is reconstructed from static analysis of `damoh-reports.component.ts`, its template, `master.service.ts`, and `get-reports.ts` as they exist today.

CONTENTS

Part 1 High-Level Design

How the component fits into ParakhWeb, what it talks to, and the shape of the ten-plus report formats it renders.

1.1 System context

`DamohReportsComponent` is a reusable report viewer dropped into the Super Admin shell wherever a `form_id` is provided. It is not routed directly — a parent screen passes `[form_id]` as an `@Input` and listens for `form_id_change` to know the resolved report title and whether that form has a report view at all.

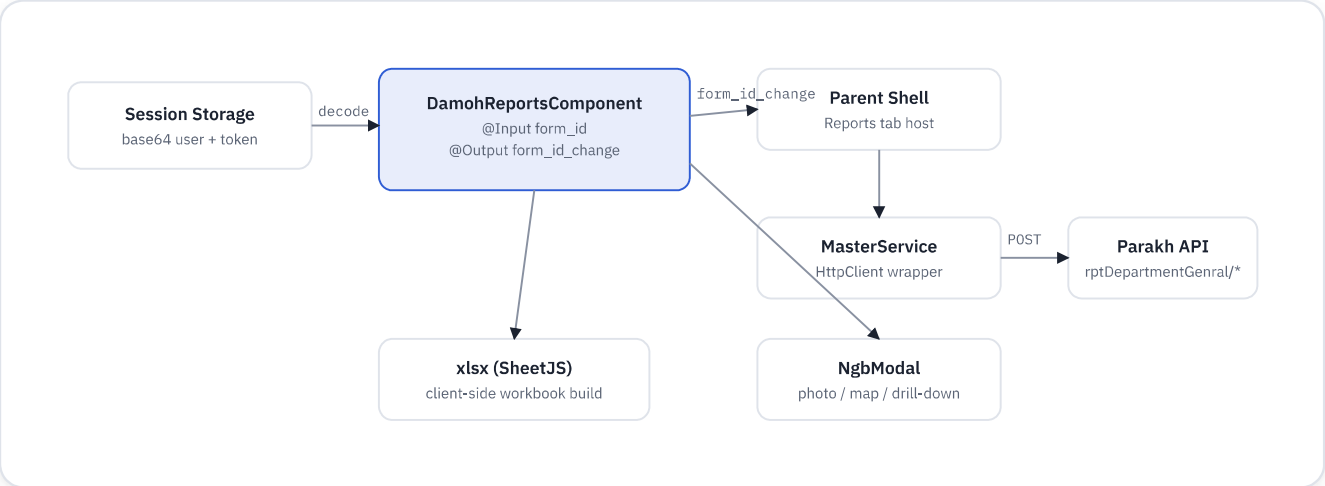

Fig 1.1 — component boundaries: session-derived auth, parent-driven form selection, service-layer HTTP, and two presentation escape hatches (modals, client-side Excel).

1.2 Component architecture

Three concerns live in one class, which is the central architectural fact about this component:

01

Report retrieval & pagination — two divergent API paths keyed off `form_id`

02

Per-form presentation — the template branches into a distinct `<table>` per form ID; the .ts side supplies form-specific field lookups

03

Excel export — a second, independent fetch-all-then-build pipeline with hand-authored sheet layouts per form ID

1.3 Report catalogue

Eleven `form_id` values are recognized via `reportTitleMap`; anything outside this map renders no title and is treated as "no report available" by `form_id_change`'s `hasReport` flag.

FORM ID	REPORT	API PATH	EXPORT SHEET
292	छात्रावास निरीक्षण रिपोर्ट	`GetDamohReport`	generic fallback
293	विद्यालय मानीटरिंग रिपोर्ट	`GetDamohReport`	generic fallback
294	विद्यालय निरीक्षण रिपोर्ट	`GetDamohReport`	generic fallback
541	कार्यालय जिला पंचायत निरीक्षण (जन विश्वास)	`GetDepartmentGeneralReportByFormID`	custom · 58 cols, 2-row header, merges
542	शासकीय उचित मूल्य दुकान निरीक्षण (जन विश्वास)	`GetDepartmentGeneralReportByFormID`	generic fallback
547	नलजल प्रदाय योजना निरीक्षण (जन विश्वास)	`GetDepartmentGeneralReportByFormID`	custom · 24 cols, 2-row header, merges
548	आंगनवाड़ी निरीक्षण (जन विश्वास)	`GetDepartmentGeneralReportByFormID`	custom · single header row
549	स्वास्थ्य विभाग (जन विश्वास)	`GetDepartmentGeneralReportByFormID`	custom · 23 cols, 3-row header, merges + col widths
550	मदिरा दुकानों निरीक्षण (जन विश्वास)	`GetDepartmentGeneralReportByFormID`	custom · single header row
552	प्राथमिक कृषि साख सहकारी समिति निरीक्षण (जन विश्वास)	`GetDepartmentGeneralReportByFormID`	custom · 49 cols (defined twice, see §3)
554	विद्यालय निरीक्षण प्रतिवेदन	`GetDepartmentGeneralReportByFormID`	custom · 32 cols, single header row

1.4 Role-based data scoping

Only the legacy `GetDamohReport` path (form IDs 292/293/294) scopes by role. `query_type` is computed from `role_id` at request time:

ROLE_ID	QUERY_TYPE SENT TO API
≤ 4	`GetGeneralReportsState`
5	`GetGeneralReportsNodal`
6	`GetGeneralReportsRo`
otherwise	`GetGeneralReportsInspector`

The generic path (`GetDepartmentGeneralReportByFormID` , used by every Jan Vishwas Abhiyan form) sends no `query_type` at all — scoping for those forms, if any, happens server-side on `user_id` / `form_id` alone.

Part 2 Low-Level Design

Field-level state, method contracts, and the two runtime sequences that matter: loading a page of results, and exporting the full dataset.

2.1 State & inputs

INPUTS / OUTPUTS

`@Input form_id: number`	selects the active report
`@Output form_id_change`	emits `{'{'}title, hasReport{'}'}` on every form_id change

CORE FIELDS

`reportList: any[]`	current page of rows
`p, itemsPerPage, totalItems`	pagination cursor (50/page)
`input: GetGeneralReport`	mutated in place and reused as the request body for every call
`tableData, columnHeaders`	nested drill-down modal state (form 542 only)

2.2 Lifecycle sequence

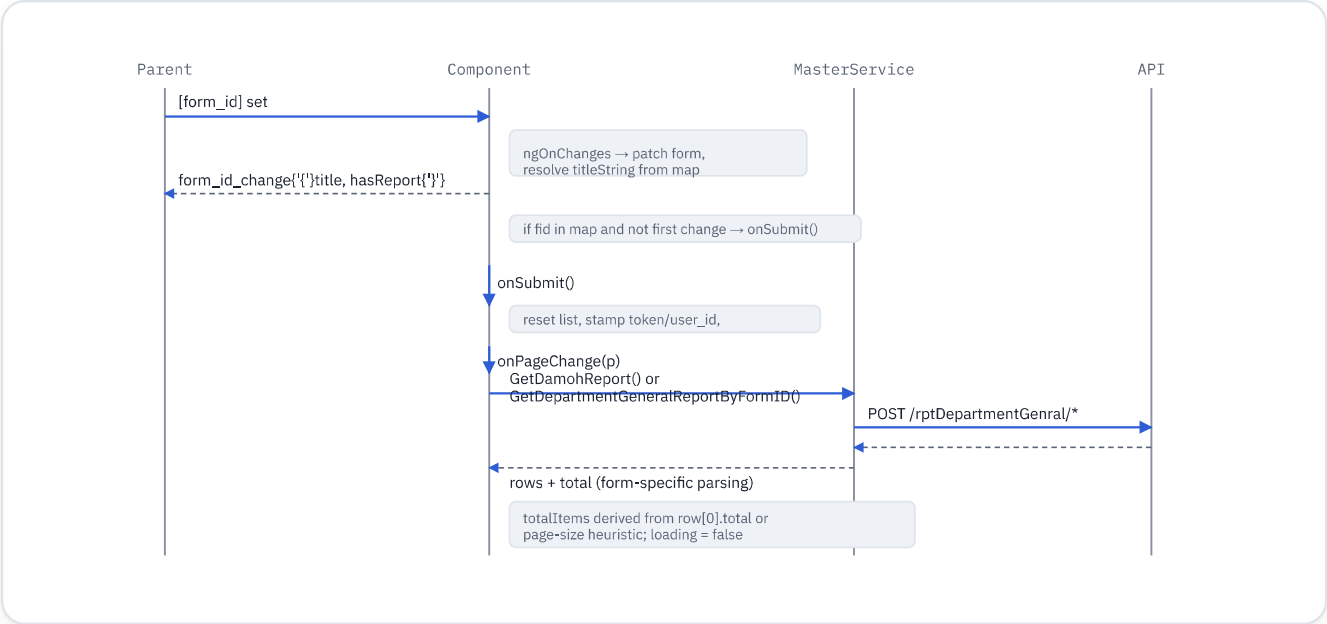

Fig 2.1 — from parent-driven `form_id` change through to a rendered page of results.

2.3 Pagination logic

Pagination is **not** uniform across the two API paths, which is easy to miss when reading `onPageChange` in isolation:

- **Legacy path (292/293/294):** `totalPages` reads `reportList[0]?.total` — the backend is expected to embed a running total on every row.
- **Generic path (all Jan Vishwas forms):** `totalItems` is inferred client-side — if the page came back full (`length === itemsPerPage`), the component assumes at least one more page exists (`(p+1) * itemsPerPage`); otherwise it back-computes the true count from the short final page. This means `totalPages` can overshoot by one page until the user actually reaches the end.
- **Range window:** `paginationRange()` shows ±2 pages around the current page (delta = 2), independent of which path produced `totalPages` .

2.4 Method reference

`onSubmit()`

`(): void`

Guards on `form.value.form_id` , resets `reportList` , stamps `token` / `user_id` / `form_id` onto `input` , then delegates to `onPageChange(this.p)` for the actual fetch.

`onPageChange(page)`

`(page: number): void`

Bounds-checks against `[1, totalPages]` , computes `offset` , then branches on `form_id` : legacy forms call `GetDamohReport` with a role-derived `query_type` ; every other recognized form calls

`GetDepartmentGeneralReportByFormID` and derives `totalItems` from the response shape.

`no error branch on the legacy path`

sort(property, event)

`(property, event: MouseEvent): void`

Toggles a chevron icon class directly via `event.currentTarget.classList`, flips `isDesc`, then does an in-place client-side sort of the current page only — sorting does not refetch or affect other pages.

`direct DOM class manipulation from TS` `page-local sort only`

open / openImg / openmodal

`(content, latlng|src): void`

`open()` parses a `"lng,lat"` string into `this.lat/lng` for a map modal; `openImg()` stashes a photo path into `ph1`; both delegate to `NgbModal.open` with an identical static/non-keyboard config. `openpdf()` is a third variant that just does `window.open` on a document URL instead of a modal.

getNestedTable(content, quesId, ansId)

`(content, quesId: number, ansId: string): void`

Fetches a per-row drill-down table (used by form 542's stock-register detail) via `GetNestedTable`, JSON-parses the `answers` column of every returned row, and derives the modal's column headers from the keys of the first row before opening it.

`column headers assume row 0's keys are representative`

getSingleValueofMultipleHeaders(item)

`(item): string`

Four Jan Vishwas forms (548, 549, 550, 554) store the same logical field under up to seven numbered keys (e.g. `आँगनवाड़ी_1` … `_7`) because the source form allowed repeatable entries. This picks the first non-null, non-`'-'` value — effectively "the one the field officer actually filled in."

2.5 Excel export pipeline

Export is a second, independent data path — it re-fetches everything rather than reusing `reportList`, because the on-screen view is paginated at 50 rows but export needs the full dataset.

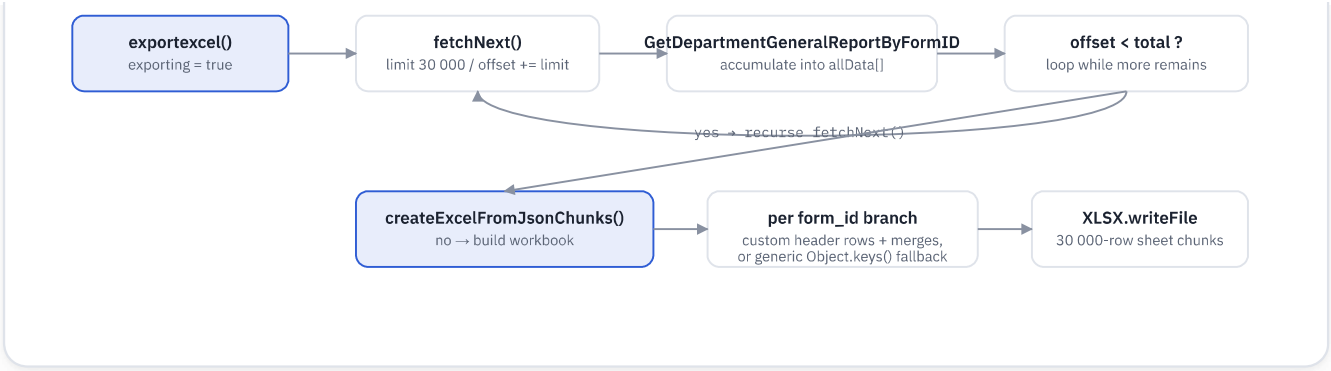

Fig 2.2 — export recursively drains the API in 30 000-row batches, then rebuilds a form-specific worksheet layout entirely on the client.

SHEET LAYOUT STRATEGY

Inside `createExcelFromJsonChunks`, each recognized `form_id` gets a hand-built `wsData` (array-of-arrays) with its own header row(s) and `ws['!merges']` definitions to reproduce the grouped/rowspan headers seen on screen — e.g. form 541 merges 9 header groups (वीबीजीरामजी, 15वें वित्त, संबल, पेंशन, …) across 58 columns; form 549 nests three header rows with 22 merge regions plus explicit `!cols` widths. Any `form_id` not given a branch falls through to a generic `Object.keys(chunk[0])` header — raw field names, no Hindi labels, no merges.

Output is chunked at 30 000 rows per sheet (`Sheet1`, `Sheet2`, …) to stay under Excel's per-sheet practical limits, and the filename is chosen from a second, separate `if/else` on `form_id`.